TECHNICAL RESEARCH / 2026

后训练工具与框架工程调研

面向自研算法：自由度、复杂度与训练效率

2026-10-10 · 大语言模型 / 视觉语言模型 / Agent / 具身扩展

研究问题

实现自己的后训练学习算法时，应当掌握哪些代码，复用哪些训练与生成部件，又如何保持高效执行？

选择	按算法改动的位置，收敛完整技术路径
拆解	追踪主循环、数据契约、损失与同步实现
验证	分清数学语义、加速路径与运行约束

证据范围

本次对核心框架下载并阅读了固定 tag 或 commit 的训练循环、损失、轨迹接口、权重同步与异步消费代码；外围工具采用官方文档和代表实现。没有运行 GPU 训练，因此文中的优先级是工程选型判断，性能机制不等于统一条件下的速度排名。源码快照、扩展入口与一手来源列于报告后部。

目录与阅读导航

先看第 1、4、5 章完成初步选型，再按第 3、10 章进入对应源码。第 11、12 章提供版本边界与可点击的一手来源。

1. 先给出可执行的选择 . . . 3
2. 整个工具栈的层次 . . . 4
3. 核心框架逐项读代码 . . . 6
4. 什么叫“能够定制每个算法细节” . . . 11
5. 高效与自由同时成立的工程条件 . . . 13
6. 周边工具应该怎样使用 . . . 15
7. 其他重要路线与维护状态 . . . 17
8. 建议你保留的最小算法架构 . . . 18
9. 从实现到选择：一套具体推进顺序 . . . 19
10. 源码阅读路线 . . . 20
11. 本次审查的版本边界 . . . 21
12. 一手来源与可复核链接 . . . 23

如何理解本报告

“优先验证”是基于代码结构与研究需求的判断。GPU 数量、目标模型和算法细节未知，报告不提供未经同条件实验验证的速度排名。核心项目读到实现，外围项目按官方资料与代表代码定位；两种证据深度在正文中区分。

正文的 [Sxx] 可跳转到来源索引；来源索引中的链接可打开仓库文件或官方文档。固定 SHA/tag 用于复核本次实现，main 与文档页标示滚动资料。

1. 先给出可执行的选择

1.1 你的默认起点

在模型规模和集群资源尚未确定、创新主要位于语言策略目标、优势估计、采样与多轮交互的前提下，我会把 SkyRL 的 FSDP2 + vLLM 路径作为第一轮完整原型的优先候选。自己的算法模块保留纯 PyTorch 实现，通过明确的 batch 与 generator 接口接入现有运行时。这是建议的验证顺序，而非已经测出的普遍最优框架。[S01][S02][S03]

理由很具体：SkyRL 有独立的 GeneratorInput/GeneratorOutput、优势估计和策略损失注册表，以及可覆写的 trainer 阶段；官方 on-policy distillation 示例用 67 行代码展示了替换奖励阶段、优势估计和 trainer 的完整入口。FSDP 路径使用 Hugging Face 模型包装，便于从熟悉的 nn.Module 开始。[S02][S04][S05]

如果新算法需要新 Q/critic、target network、多个优化器、不同的更新频率或跨迭代经验重用，就先决定自己是否持有 learner 主循环。可以继续复用框架的训练 worker、推理服务和权重同步；不必重写整套系统。真正需要自己掌握的是算法状态、数据寿命与更新规则。

对于你已有的强化学习研究经验，这一点比框架内置多少种 PPO/GRPO 变体更有意义。类似自定义熵、价值算子或离策略估计的创新，通常会跨越普通 reward callback 的边界。

1.2 按需求收敛，而不是同时安装所有框架

当前目标	优先验证的完整路径	选择依据与边界
SFT、DPO 或离线偏好目标	TRL + Transformers + PEFT（需要时）	启动快，标准数据和 reference 流程完备；自定义损失仍需审计 fused 路径
语言策略 loss、advantage、多轮 agent	SkyRL + FSDP2 + vLLM	算法与生成接口明确；先沿同步路径验证
大模型/MoE，已经接受 Megatron	slime + Megatron + SGLang	算法连接层很薄，loss/advantage/generation hook 直接
明确需要 SGLang、HF/FSDP 或版本相关消费策略	Miles + 选定训练后端 + SGLang	多后端与可替换 DataBuffer 有价值，整体运行层更丰富
已有 HF + DeepSpeed 积累	OpenRLHF + DeepSpeed + vLLM	Experience 和 actor/critic 路径直观，维护小 fork 往往可行
团队平台、多后端、多算法配方	verl V1	生态和运行时能力广，数据与控制抽象更多
多任务混采、多角色和异构资源部署	ROLL	Worker/Strategy 可替换；需要理解 pipeline 与调度器
异步与 agent 接入就是研究主题	AReaL / SkyRL / Miles / NeMo RL	先读版本、队列和数据消费语义，再选异步机制

当前目标	优先验证的完整路径	选择依据与边界
新多网络学习规则、target /replay/复杂优化	自己的 learner + 可直接操作的模型 + FSDP2/DeepSpeed + 成熟 rollout 服务	最大化算法控制；适配、恢复和同步仍有工程成本

上述路线分别依据对应项目的代码与官方接口。完整框架之间通常选择一个主运行时，底层训练和推理部件则按已有兼容路径组合。[S06][S08][S13][S17][S21][S26][S31][S34]

1.3 “最简单”要拆成三种成本

启动简单，意味着少量配置就能复现实验。TRL、LLaMA-Factory、Axolotl、ms-swift 擅长降低这一成本。

改算法简单，意味着沿着一条短调用链找到数据、概率、损失、反向传播和更新。SkyRL、slime 的明确扩展点，以及自己持有的 PyTorch learner，往往更贴合这一需求。

大规模运行简单，意味着框架替你处理分片、权重重分布、长尾 rollout、故障恢复与资源切换。verl、NeMo RL、AReaL、Miles、ROLL 等投入更多运行层代码来解决这些问题。它们增加的概念可能正是后期省下的系统工作。

因此，入口脚本的行数不能单独决定选型。一个很短的 driver 仍然可能依赖复杂的 Megatron、Ray、推理引擎和模型映射。

2. 整个工具栈的层次

2.1 从算法到硬件，共六层

层次	负责什么	代表工具
算法与实验层	reward、advantage、loss、critic、teacher、更新规则、评估	自有算法模块；TRL、SkyRL、slime、verl 等的算法实现
数据与运行层	rollout、环境、多轮轨迹、buffer、资源布局、版本发布	SkyRL、verl、OpenRLHF、AReaL、Miles、ROLL、NeMo RL
模型与训练执行层	forward/backward、状态分片、TP/PP/EP/CP、优化器	PyTorch/FSDP2、DeepSpeed、Megatron-Core、TorchTitan、VeOmni、NeMo AutoModel
推理生成层	batching、KV cache、prefill/decode、sampling、热更新	vLLM、SGLang；JAX/TPU 路线另有对应后端
参数化与 kernel 层	LoRA/QLoRA、attention、融合损失、低精度、通信 kernel	PEFT、bitsandbytes、Liger、FlashAttention、FlashInfer、Transformer Engine、DeepEP、torchao
数据质量与观测层	数据加工、验证器、评估、日志、profile、checkpoint	Datasets/DataTrove、distilabel、Math-Verify、lm-eval、W&B/Trackio/SwanLab、PyTorch Profiler、Nsight

一个项目可以覆盖多层，这张表表示主要职责。FSDP2、DeepSpeed 和 Megatron 不能代替完整 rollout/奖励/数据消费闭环；vLLM 与 SGLang 也不负责训练模型的反向传播。[S47][S50][S51][S54][S56][S57][S58][S72][S73][S74][S75][S78][S79]

2.2 DeepSpeed 为什么应当单独评价

DeepSpeed 的价值在训练执行和内存管理层。deepspeed.initialize 接收用户模型、优化器或优化器工厂，再提供 engine.backward/step 等接口。你可以保留自己的目标函数与训练循环，使用 ZeRO、混合精度、offload 和分布式优化器。[S47][S48]

它与 FSDP2 在状态分片方面有重叠，与 Megatron 的模型并行能力又有不同侧重。DeepSpeed-Chat 则是在这个底层之上实现的一套具体 RLHF/PPO 示例，有 actor、reference、critic、reward 等角色。不能用 DeepSpeed-Chat 的算法形态概括 DeepSpeed 的全部能力。[S49]

同一个算法的学习质量取决于数值、采样、损失归一化与优化器行为是否忠实实现；DeepSpeed 的分片和执行机制主要影响能否运行及系统成本。比较学习效果时，应先确认这些算法条件一致。

2.3 可组合与不可随意组合的边界

合理的组合是一条已经贯通的路径：模型实现、训练后端、权重导出/同步、推理引擎、轨迹数据、算法更新。最初优先采用框架提供的 lockfile、镜像和完整示例，之后再替换一个部件。

FSDP2 与 DeepSpeed ZeRO 通常是同一个训练模型的不同状态管理路线，不适合不加设计地双重包裹。vLLM 和 SGLang 通常也是同一生成角色的候选后端。Ray 负责进程与资源编排，torch.distributed/NCCL 负责张量通信，两者属于不同层次。

“支持模型 A”“支持 LoRA”“支持 Megatron”“支持某种权重传输”也不能推出它们的任意交叉组合都已实现。模型参数命名、MoE expert 布局、量化后重排、LoRA merge 和训练/推理的分片形状，都可能构成组合边界。[S20][S54][S56]

2.4 训练范式决定需要哪些角色

范式	主要数据与计算	对研究底座的要求
SFT / continued training	给定序列上的监督目标	模型、mask、packing、训练执行；不必引入在线 rollout
Reward / preference modeling	偏好对、等级或过程标签	评分 head、pair/group batch 与验证方式
DPO / IPO / KTO / ORPO 等离线目标	固定偏好数据；部分目标需要 reference	目标和 reference 路径应独立检查，不能假定这些方法公式等价
PPO / GRPO / REINFORCE 类在线学习	当前策略生成、评分、估计、更新与发布	生成引擎、轨迹契约、old/behavior 概率、actor；按算法增加 critic
自定义离策略 / 多网络学习	历史经验、Q/target、多种更新规则	可控 learner、长期 buffer、多个 optimizer 与完整 checkpoint
在线蒸馏与 agent 学习	student 或 teacher rollout、工具/环境、多轮信号	teacher 输出粒度、轨迹/credit 语义、环境并发与终止定义

这张表用于识别工程需求，不代表某个框架已支持所有列举方法的全部配置。TRL、AReaL、NeMo RL 等各有现成实现，选择仍应落到具体目标和调用路径。[S08][S09][S12][S31][S35]

3. 核心框架逐项读代码

3.1 SkyRL：我会优先试的通用研究路径

核查版本：skyrl-v0.4.0（2026-10-01）及当日固定 commit。统一目录已经进入 release，旧教程中的 skyrl-train 路径可能与当前 skyrl/train、skyrl/backends 不同。[S01]

训练控制主体位于 skyrl/train/trainer.py 的 RayPPOTrainer。生成与算法分离：GeneratorInterface.generate 接收 GeneratorInput，输出原始 prompt token、response token、reward、loss mask、rollout logprob，以及可选的组标识、多模态数据、MoE 路由和 sampler support。策略损失和优势估计有独立 registry。[S02][S03][S04]

这为三种常见研究改动提供了清晰入口：新 advantage 注册一个估计器；新常规策略目标注册 policy loss；新环境或多轮逻辑实现 generator。超出这些接口时可以覆写 trainer 阶段，而不是马上修改所有分布式 worker。

官方 on-policy distillation 示例尤其值得阅读：OnPolicyDistillationTrainer 覆写 apply_reward_kl_penalty，利用 student 与 teacher 的 sampled-action logprob 差产生 token 信号，再注册 no_op advantage。它说明能改算法阶段；它实现的是特定采样估计，并不自动代表任意全词表蒸馏目标。[S05]

FSDP2 路径包装 Hugging Face 的因果语言模型/视觉语言模型。Megatron 路径提供大模型并行能力，但本次读取的 MegatronCriticWorkerBase 直接未实现 critic；需要 actor-critic PPO 时应检查 FSDP 路径。VLM 还存在 padding removal 与 sequence parallel 的组合断言。模型可加载并不意味着每种优化都可启用。[S06][S07]

完全异步 trainer 有独立的约束与生命周期。其 max_staleness_steps 主要参与准入/容量控制，不能直接当作每条完成轨迹的严格最大版本年龄。你若研究严格离策略窗口，需要检查实际消费规则。

部署成本需要单独评估：0.4.0 release 的 Linux GPU 依赖锁定 vLLM 0.30.0、PyTorch 2.13.0 和 CUDA 13.0 路线，部分扩展使用项目提供的预构建 wheels，Python 要求也较新。这里推荐的是算法接口结构；实施时应从这个 release 对应的完整环境开始。release 与固定 main 的 generator、OPD 示例等文件一致，但 trainer、async trainer 与依赖文件已有变化，不能混写为同一快照。[S01][S03]

我的判断： 对语言策略目标和 agent rollout，SkyRL 在明确接口与复用系统部件之间取得了适合研究的平衡；它仍然包含 Ray、分布式 worker 与同步层，单卡离线实验未必需要整套运行时。

3.2 TRL：SFT/DPO 基线优先，在线 RL 看生命周期

TRL 的主要价值是与 Transformers、Datasets、Accelerate、PEFT 等生态紧密衔接。SFTTrainer、DPOTrainer、GRPOTrainer 给标准数据和目标提供了现成路径。当前 _BaseTrainer 继承 Transformers Trainer，因此优化步、梯度累积、callback 和 checkpoint 生命周期仍主要来自上游。[S08][S09][S12]

修改入口并不只有 reward_funcs：SFT 可用 compute_loss_func；当前 DPO 在 _compute_loss 内处理 log ratios 与偏好目标；GRPO 有 rollout_func、生成评分阶段和 _compute_loss。源码里的 generation refresh、microstep 与 optimizer global_step 是不同的时钟。新 replay 或 update-to-data ratio 需要读 _prepare_inputs 的缓存、shuffle、拆批与再生成规则。[S08][S09][S12]

权重同步位于 trl/generation/vllm_generation.py。FSDP2 用 full_tensor 收集参数，ZeRO 使用 gather context，PEFT 还需要 merge/unmerge 和名称处理；server 与 colocate 模式的传输不同。只有主 rank 发送数据，也不意味着其他 rank 可以跳过 collective。[S10]

一个实质性效率边界： 本次快照常规的 SFT 自定义 compute_loss_func 分支使用完整 logits forward，并另做一次无梯度 fused forward 计算指标；默认路径则可直接使用 fused LM head。该结论不包含前置的 DeepSpeed Ulysses sequence-parallel 专用分支。换损失后应检查实际进入的路径，不能直接继承默认吞吐与显存预期。[S09]

TRL 当前已有实验 AsyncGRPOTrainer，不能再概括为只支持同步。它提供独立 RolloutWorkerProtocol 和 WeightTransferProtocol，但官方明确列出分布式只支持 FSDP2、并非 DeepSpeed ZeRO，且实验接口可以变化。普通 GRPO 的 vLLM server 模式与异步 trainer 也不是同一件事。[S11]

我的判断： 离线 SFT/DPO 用 TRL 建基线很合适；常规 GRPO 也可快速起步。若学习过程经常超出其数据刷新、组构造与单 Trainer 更新生命周期，保留自己 learner 或使用更明确的 RL 运行时会更清楚。

3.3 slime：Megatron + SGLang 的薄算法连接层

核查版本 v0.4.0，2026-10-09。 当前 slime 已移除 FSDP，最新版本还把同步和 fully-async 入口统一到 train.py。不要沿用旧版的后端与入口说明。[S13]

实际主链为 train.py、RolloutManager.generate、batch_builder.convert/split_by_dp、MegatronTrainRayActor.train_actor、loss_function。入口脚本确实很短，关键价值在于把生成、优势、目标和偏差修正分别开放。[S14][S15]

自定义 loss 的签名接收 args、batch、可微 logits 和 sum_of_sample_mean reducer；自定义 advantage 接收 args、整批 rollout_data 并原地填写优势/returns。另有 custom generation 与 custom TIS 接口。相比只能取得 selected-token logprob 的普通 policy-loss registry，这种 logits 入口对某些熵和目标研究更直接；但 TP 下 logits 可以是词表分片，并非每卡都有完整分布，仍需遵守并行算子的契约。[S15]

Sample 携带原始 token IDs、response_length、loss_mask、rollout_log_probs 和权重版本。一次 agent 执行被拆为 compact/subagent 多个训练样本时，rollout_id 的一致性会影响样本加权。CP 下局部 logprob 与完整 mask 不同形，外层 Megatron 还会施加微批/全局 batch/并行缩放；自定义 loss 不宜重复归一化。[S15][S16]

默认 fully-async 路径有背压、年龄指标与过滤扩展，但本次审查没有看到 Miles 式默认显式硬年龄门限。记录 staleness 与执行严格 age gate 是不同能力。

我的判断： 已经接受 Megatron 和 SGLang，且研究目标主要是策略梯度、优势、熵、采样或 agent 轨迹时，slime 很值得优先。它的薄层不会消除 Megatron 模型接入、并行拓扑和权重映射成本。

3.4 Miles：可替换异步消费策略，更多运行能力

核查版本 v0.1.2，2026-10-09。 Miles 从 slime 分叉并继续演进，当前已有 Megatron、FSDP 和 TorchTitan 三条训练路径。公共实现迁至 miles/backends/training_utils，旧源码路径可能不再存在。[S17]

objective.py、hub/losses.py、hub/advantages.py 组织公共目标；WeightUpdater 统一暂停引擎、传送权重桶、结束更新、发布版本和恢复生成。协议层对 colocate、packed broadcast、LoRA 等组合有显式限制，因此不能仅按功能表推导任意组合。[S18][S20]

它最值得研究的接口之一是 custom_async_data_buffer_path。FullyAsyncRolloutFn 把后台持续生产与每一步消费哪些组分开，buffer 的 put/get 可看到 current_version 和 trainer_model_id。适合做优先级、样本重用、难度混合与版本相关采样。[S19]

默认 buffer 仍然消费后移除样本，真正 replay 需要自己的实现。max_weight_staleness 未设置时不启用硬过滤；过滤在消费时计算，避免把进入队列时的年龄误当训练时年龄。权重版本可绑定 token span，一条生成可能跨多个版本。[S19]

Miles 也不是 slime 的逐项接口超集：本次读取的优势估计派发仍以 if/elif 组织，没有确认到与 slime 相同的外部 custom-advantage hook。修改公共 advantages 文件依然可行，但要以当前源码为准。[S18]

我的判断： 对明确使用 SGLang、需要 HF/FSDP 训练或研究异步 buffer 的项目很有吸引力；如果多策略、运行恢复和多后端都暂时用不到，其整体复杂度可能超过你的第一版需要。

3.5 OpenRLHF：熟悉 DeepSpeed 时很自然

核查版本 v0.11.2。 主路径为 BasePPOTrainer.train_step、RemoteExperienceMaker.make_experience_batch、ActorPPOTrainer.training_step，再由 DeepSpeed 执行 backward/step，最后同步至 vLLM。[S26][S27][S28][S29]

Experience 是直接的 dataclass：完整序列通常是 B×T，action mask、policy/value/advantage 是 B×A，episode reward 是 B。新 baseline/advantage 主要看 experience_maker.py；新 surrogate、clip、IS 与 reduction 看 models/loss.py；entropy、辅助目标和 optimizer 接线看 ray/ppo_actor.py。[S27][S28][S29]

它区分当前训练 logprob、固定 old logprob 与真实 rollout logprob，处理训练推理不一致和离策略修正。组奖励在重排后按 prompt 标识恢复，说明 group 统计不能随意在每个 microbatch 或 rank 内独立计算。[S27][S28]

异步路径使用生成/训练 actor、有界队列和 slot 背压；普通模式以锁协调广播，partial rollout 模式暂停/恢复生成。有界队列对领先程度施加约束，但不等同于逐样本版本过滤。ZeRO3 同步还需 GatheredParameters 等上下文。[S29][S30]

我的判断： 已有 HF+DeepSpeed 经验的研究者，维护这几个明确文件的小 fork，可能比切换整套并行训练生态更快。它并非每种算法部件都有通用插件协议，但修改点通常容易定位。

3.6 verl V1：功能与后端广度，付出更多抽象成本

核查版本 v0.9.1。 v0.9 系列默认 V1 trainer，统一 sync、colocate_async 与 separate_async。旧 RayPPOTrainer.fit 教程不足以解释当前默认路径。[S21]

当前链路为 TaskRunnerV1、get_trainer_cls(mode)、PPOTrainer.fit/step/_step_once。每步组织 sample、reward、balance、old/reference logprob、value、advantage、critic 与 actor 更新。数据通过 TransferQueue 存储，KVBatchMeta 持引用，worker 使用 TensorDict；优势阶段还借用 DataProto 再恢复 nested tensor。[S22]

register_adv_est 与 register_policy_loss 适合共享算法算子。普通 worker policy-loss 接口主要传 selected-token logprob、advantage、mask 与权重；若目标需要任意 hidden state 或完整 logits，需要进一步改 worker/model-output 层。[S23][S24]

V1 支持自定义 ReplayBuffer/sampler，异步消费还有 drop 与 wait 两种策略。wait 在在途 prompt 达到阈值时暂停推进，等待慢样本完成，避免直接丢弃这一类数据。其 off-policy 计数约定与别的框架并不相同；同样配置为 2 不一定表达同样的年龄。[S25]

默认 buffer 还涉及 prompt 生命周期、结果物化和清理。它的名字包含 replay，并不说明默认行为就是长期、可反复采样的经典离策略经验池。

我的判断： 如果需要一个团队共享、多后端、多配方、长期扩展的平台，verl 很强；若目标是最短时间理解并验证一个新学习规则，V1 的控制与数据抽象需要额外阅读成本。

3.7 AReaL、NeMo RL、ROLL：系统要求明确时再提高优先级

AReaL、NeMo RL 和 ROLL 都值得进入候选集，但它们最突出的价值在于异步、资源组织、模型规模或多种训练任务。对第一版研究代码，先明确自己会用到哪一项系统能力。

AReaL 2.0：已有 agent 与异步学习的系统底座。 当前架构拆出 training、inference、agent 和 weight-update 服务。RolloutWorkflow.arun_episode 以及较高层的 run 接口负责环境执行；算法主路径进入 PPOTrainer.train、PPOActor._compute_advantages、_ppo_update，再由 engine.train_batch 接收 loss 函数。接入现有 agent 容易，与任意新学习规则的实现成本应分开评价。[S31][S32][S33]

实读 actor 中有 token/turn 粒度 GAE、截断 bootstrap、token_rewards 与 old/proximal/behavior 概率区分。token_rewards 也有模式约束：例如某路径要求一轮内一致的奖励，而任意稠密 token 信号走另一配置。多教师 OPD 和权重发布 controller 提供了进一步扩展材料。若创新涉及这些细节，直接读 actor/gae/loss 与 TrainEngine；早期 AReaL-lite 的极简印象不足以描述今天的系统。[S32][S33]

NeMo RL：损失契约很清楚，运行层也更丰富。 PolicyInterface 暴露 logprob/top-k、train 和状态准备；LossFunction 显式区分 TOKEN_LEVEL/SEQUENCE_LEVEL、LOGIT/LOGPROB/OPD_FULL/DISTILLATION 等输入，并传入 global_valid_toks/global_valid_seqs。把分布式归一化写进协议，是很值得借鉴的设计。[S34][S35]

其实际 GRPO 有 legacy、SingleController、TransferQueue 等路径；蒸馏又区分采样 token、teacher top-k、full OPD 和多教师路由。全分布 OPD 的部分能力要求 SingleController，不能沿旧入口推断可用。当前所读 async GRPO replay 路径明确排除 SGLang，虽然项目在其他路径支持 SGLang。复杂 MoE、教师模型和 NVIDIA 并行栈是它值得优先评估的场景，代价是依赖与运行协议更广。[S34][S35][S36]

ROLL：多角色与异构任务平台，loss 入口有实质自由度。 核查 v0.4.0，主结构为 Pipeline → Cluster/Ray dispatch → Worker → Strategy；rollout 另经 DynamicSamplingScheduler、Router 和 ReplayBuffer。可用 actor_train.worker_cls 指定自定义 ActorWorker，覆盖 loss_func(data, output_tensor)，同时接管 entropy、KL、样本权重和 surrogate。输入是前向 logits，其 TP 布局应通过 Strategy 的 logprob/entropy 算子处理。[S76]

更改 advantage 要进入 compute_advantage 的固定分支或替换 pipeline 调用；更改更新规则则跨 BaseActorWorker.train_step 的 epochs/batch 与 Strategy 的 microbatch/backward/optimizer。自定义 rollout 有 user_defined_rollout_loop_cls 和 process_new_prompt(context) 入口。v0.4.0 的 RLVRPipeline 已实现 async_pipeline，训练时推理/评分可以继续，下一轮权重发布前才暂停采样；README 中旧的未实现说明落后于这段源码。[S76]

ROLL 的 ReplayBuffer 按 sampling_start_step 和跨步窗口组织生成数据，消费时移除，默认不提供长期反复经验重用。这个年龄也不是逐 token 的权重版本。它更适合明确需要多 domain 混采、多奖励集群和复杂资源布局的项目，与 verl 一起作为平台候选。[S76]

这些项目提供的异步机制不能只按“支持/不支持”比较。后文按生产者准入、队列容量、消费时过滤与权重版本解释关键差异。

3.8 prime-rl、verifiers、rLLM 与 Agent Lightning

prime-rl 与 verifiers 的组合值得环境和 agent 研究者关注：环境、评分和训练消费可以分离，orchestrator 的 Algorithm.score_episode/score_group 决定训练信号，trainer 的 CustomLoss 处理优化目标。当前 CustomLoss 输入围绕每条 sample 的 selected-token logprob，任意 hidden state、全 logits 或跨 sample 目标需要扩展；训练模型又有 native implementation 范围。其队列出队前 _drop_stale 实施硬年龄过滤，适合比较异步数据消费语义。[S37][S38]

rLLM 的优势是 Episode/trajectory、组数据、credit 与多 backend 接入。其当前能力矩阵明确指出统一 API 不保证所有后端具有同样能力：例如某些路径没有 critic、某些 reward/credit 模式受限。若创新重点在 agent 数据与信用分配，它很有价值；若每个优化器和目标细节都要自己控制，仍需进入后端。[S41]

Agent Lightning 对已有 agent 训练接入有吸引力。当前代码结构包含 server、controller 与 verl 集成；轨迹 adapter 对 exact token prefix 和 observation mask 的处理值得研究。其共享 GPU 异步流程要暂停/排空生成、更新再恢复，未完成 agent group 可跨训练步保留；不应把这种交错执行写成同一 GPU 同时训练与生成。学习目标的深层自由度仍取决于训练后端。[S40]

3.9 TorchTitan / TitanRL 与 torchtune

TorchTitan core 是 PyTorch 原生大模型训练底座，当前已经包含 SFT 和实验 TitanRL。TitanRL 将 Trainer、Generator、Controller 与数据结构分开，并尝试在训练和 vLLM 生成间复用模型定义及内核。对研究训练/推理数值一致性、模型结构和系统实现的人，它值得认真看。[S42][S43][S44]

其 Trainer.forward_backward 与 optim_step 清楚分开，service 可以独立复用；完整运行时还涉及 Monarch、TorchStore，当前安装指引使用匹配的 PyTorch nightly 与 vLLM 构建。RL 的模型支持明显小于 TorchTitan core，且 README 将该表中的支持定义为代表性 debug 模型完成两步端到端运行；PP>1 路径直接被限制。统一模型与 batch-invariant 模式也可能付出吞吐代价。[S43][S44]

本次读到的 ChunkedLossWrapper 会在 LM-head 边界 detach hidden、逐块计算与反传，再合并 decoder 梯度。当前梯度桥接的 backward 忽略外层 grad_output，并注明返回 loss 后再做张量运算存在限制；连 loss * 0.5 这样的外层常数缩放都需要审查。可按块分解的系数与目标可以放入 inner loss；跨块非线性耦合、高阶梯度等仍可能需要关闭或改写这一特定优化路径。[S45]

torchtune 的显式 PyTorch recipe 仍然是很好的阅读材料，SFT、DPO、PPO、GRPO 等路径容易追踪。但官方已经明确停止积极维护。把它作为代码设计参考合理，作为 2026 年新项目的长期默认依赖应慎重评价维护成本。[S46]

4. 什么叫“能够定制每个算法细节”

4.1 六种自由度与对应入口

自由度	典型研究改动	实际需要的控制
目标函数	新 surrogate、entropy、divergence、蒸馏目标	当前 logprob/全 logits/hidden、mask、可微图、reducer
估计器	新 advantage、baseline、return、credit	完整组/轨迹、value、reward、跨 rank 统计
数据寿命	replay、优先级、动态采样、课程	buffer 写入/采样/删除、group ID、版本、数据权重
模型状态	Q head、多个 critic、target/teacher、ensemble	模型创建、独立 optimizer、checkpoint、同步策略
更新规则	actor/critic 比例、UTD、内层优化、元梯度	learner 主循环、backward 顺序、step/skip/accumulation
执行调度	异步、动态资源、partial rollout	准入、背压、权重发布、请求生命周期、恢复

框架里能传 custom loss，通常只覆盖第一行的一部分。只有一个 selected-token logprob 输入，也不能自动实现需要完整词表概率或任意 hidden state 的目标。

4.2 针对常见创新，选择最小修改层

新的策略梯度/组内优势。 优先使用 registry 或 custom advantage；保持原有 batching、actor forward 和训练后端。重点核对 group 统计是否跨 rank、是否包含过滤样本，以及标准差/长度归一化的定义。

新的熵或正则。 先说明数学对象是全分布熵、采样估计还是某种代理。只用生成 token 的 logprob，通常不能直接得到精确全词表熵。若采用 chunked/vocab-parallel 实现，需要保证 reduction 与梯度正确，而不是为了节省显存悄悄换了目标。

新的 critic/Q、target 或价值算子。 把模型、optimizer、目标更新、checkpoint 和数据字段一起设计。已有 value worker 可以复用的前提是接口、动作表示与训练时序符合需求。SAC/BAC 一类多网络设计不能靠替换 GRPO loss 自动得到。

新的 replay 与异步学习规则。 PPO 内部重复几个 epoch、缓存上一批数据、生成队列和跨策略经验回放是四种不同机制。明确样本可以使用几次、如何重新加权、旧概率从哪里来，以及失效/删除规则。

新的 teacher/student 学习。 区分教师生成数据后的离线 SFT、在线采样动作的 logprob 差、top-k 分布近似和全词表 KL。它们需要不同数据与前向开销。SkyRL 的短 OPD 示例适合作为一个具体实现起点，不能替代这种定义工作。[S05]

4.3 需要保留的轨迹契约

下面是建议由你自己定义并长期保持的字段，而非某个框架的现成 API。

字段/结构	为什么必要
原始 token IDs 与每轮输入	保证训练评分的序列确实对应生成行为，避免重新 tokenize 漂移
attention mask、loss mask、segment/position 信息	有效上下文、需要学习的动作和 packing 隔离不是同一个掩码
behavior logprobs	真实采样策略概率；明确 temperature/top-p/top-k 变换
old/current/reference/teacher logprobs	固定旧策略、正在更新策略与参考/教师承担不同职责
group/episode/turn/sample IDs	组估计、多轮 credit、动态过滤、分布式重排与样本重用
reward、advantage、return、value	保存定义、形状、dtype 和 stop-gradient 约定
policy version 或 token-span version	异步/partial rollout 可跨版本；单一版本号有时不够
terminated 与 truncated	环境终止与 token/time budget 截断可能需要不同 bootstrap
sampler support / MoE routes（需要时）	截断采样分布与路由重放的数值对齐
数据权重、采样次数、来源与过滤原因	replay/curriculum 与跨方法公平比较

SkyRL、slime、Miles、OpenRLHF 与 verl 都能看到其中若干字段的具体实现。保留自己的逻辑契约，有助于迁移 backend 时检查哪些信息被丢失。[S02][S16][S19][S27]

4.4 算法自由度对照：接口能到哪里

路线	常规目标/估计器的进入方式	超出接口之后主要修改哪里
TRL	Trainer subclass、compute_loss_func、rollout/reward hook	数据刷新与上游 Trainer lifecycle；多角色 learner
SkyRL	policy loss / advantage registry、generator、trainer 阶段	model worker、数据字段、更新主循环
slime	custom loss 取得 logits/reducer；custom advantage/generation	Megatron actor/schedule、Sample 与运行主循环
Miles	公共 loss/advantage、custom DataBuffer、生成函数	backend worker、多模型/更新协议；并非每项都有 registry
OpenRLHF	修改 loss、experience maker、actor worker	DeepSpeed 接线、角色与训练循环
verl V1	estimator/loss registry、buffer/sampler、worker	TransferQueue 生命周期、模型输出、trainer/worker 时序
ROLL	自定义 Worker.loss_func、rollout loop 类	advantage pipeline、Worker 与 Strategy 的更新边界
AReaL	workflow、PPOActor 估计/更新、TrainEngine loss	服务角色、数据与 learner 协议

路线	常规目标/估计器的进入方式	超出接口之后主要修改哪里
NeMo RL	明确的 LossFunction/PolicyInterface、多类输入	backend 与 SingleController/legacy/TQ 的组合边界
prime-rl	episode/group scoring、per-sample CustomLoss	native model、跨样本目标与 trainer 协议
自有 learner	直接持有模型、optimizer、数据与 step	自己维护与现有执行/生成运行时的 adapter

这里的“进入方式”表示本次实际看到的代码结构。源码开放意味着理论上可修改整个项目；真正要比较的是实现你的算法需要跨越多少边界、保留多少性能路径，以及未来维护这些改动的成本。
[S04][S08][S15][S18][S23][S27][S32][S35][S37][S76]

5. 高效与自由同时成立的工程条件

5.1 先看端到端时间

同步训练一轮大致包含生成、环境/评分、训练、权重同步和等待。流水化可以让部分时间重叠；在理想独立资源条件下，主要周期可从两个阶段的和趋近较慢阶段的时间。这个推导忽略传输、资源争用与学习分布变化，不是实测加速比。

建议至少分解记录 rollout、reward/env、policy/reference/value forward、backward/optimizer、weight sync、idle，以及 peak memory。真正比较的是达到同一个验证目标需要多少 GPU 小时，或者固定 GPU 时间下的验证分数。同时报告有效 token、优化步数和数据过滤比例。

只比较 tokens/s 可能奖励更长但无效的生成；只比较每秒 optimizer step 可能隐藏更小 batch；只比较总 wall time 可能忽略使用了更多 GPU。

5.2 FSDP2、DeepSpeed、Megatron 的取舍

FSDP2 以逐参数 DTensor 和 composable fully_shard 为基础，适合保留 PyTorch 模型与控制流。它解决分片训练，但 rollout、reference/critic、权重同步和异步消费仍由上层负责。[S50]

DeepSpeed ZeRO 为参数、梯度与 optimizer state 分片，offload 可以扩大可运行规模。它的 engine 在梯度累积与 step boundary 上有约定；若自己已按累积步数除过 loss，再让 engine 按同一累积步数缩放，就会重复缩放。ZeRO2/3 也不能照搬 DDP no_sync 的使用方式。[S47][S48]

拥有 nn.Module 不等于完整参数与梯度始终本地可操作。ZeRO3 要通过规定接口读写完整或本地梯度，当前 optimizer step 不支持 closure；FSDP2 也要区分 DTensor 的本地 shard 和 full_tensor 的 collective。自定义梯度变换、额外优化步骤与 target 更新，都必须遵守后端的状态和通信规则。[S48][S50]

Accelerate 可以辅助分布式接线，但也有具体边界：本快照 Accelerator.prepare 的 FSDP2 分支要求模型和 optimizer 一起准备，并拒绝一次调用传入多个 nn.Module；DeepSpeed 分支也拒绝一次准备多个模型。这是 Accelerate 当前这层接口的限制，不能推断原生 FSDP2 或 DeepSpeed 无法训练多网络。多 critic、Q/target 研究应先设计各模型、优化器与进程组的接线。[S53]

Megatron-Core 在 TP、PP、EP、CP 和大 MoE 执行上提供成熟部件。Megatron-Bridge 除模型转换外还提供训练入口和 recipe；forward_step 返回输出与 loss closure，pipeline scheduler 组织微批与反向传播。普通公式修改容易局部化，任意内层优化或多次反传可能触及 schedule。[S51][S52]

资源未知时，不必为了未来可能出现的规模立即承担全部并行维度。选择与当前模型、互联和目标最匹配的一条路径，再用 profile 决定下一层优化。

5.3 vLLM / SGLang 是推理后端，也是 RL 的重要协议边界

vLLM 当前有 WeightSource、TrainerWeightTransferEngine 与推理侧 WeightTransferEngine，把权重来源、控制客户端和数据传输分开。NCCL、同卡 IPC 和分片重分布等路径适用于不同部署。[S54]

原生接口降低了自持 learner 的同步接线成本，但模型名称/布局转换、所有 rank 的 collective 顺序、LoRA/量化语义、缓存与版本仍需正确处理。ModuleSource 对 FSDP 参数 full_tensor，并不等于所有模型并行布局都无需适配。

SGLang 提供磁盘、tensor/IPC、distributed group 等更新方式，以及暂停/继续、显存释放/恢复。当前权重更新 session 在 begin/end 之间写 checkpoint 格式值，结束时执行量化参数后处理，避免每个 bucket 都重复 repack。[S56]

凡是生成行为需要的新增策略参数，都必须能被推理后端表达并正确同步；仅用于学习的 critic/Q/target 参数可以留在 learner。如果 Q 参与实际生成时的动作选择，还需对应的评分/采样接入。只训练 LoRA 也不意味着同步一定只传 LoRA；不同集成可能 merge/unmerge 或收集整模。

5.4 异步不是一个布尔选项

机制	它实际约束什么	没有自动保证什么
有界队列/背压	待消费数量、生产领先程度	每条样本在训练时都不超过某版本年龄
准入窗口	新任务能否继续启动	极慢的在途任务完成时仍然足够新
消费时 age gate	训练前按版本过滤或等待	丢弃慢任务后数据分布不变
importance correction	按已知概率修正部分分布差异	任意旧数据重用均无偏、无方差代价
mid-flight 权重更新	生成与训练重叠程度	一条轨迹来自一个固定策略

本次源码看到的实现并不等价：OpenRLHF 主要是 queue/slot 背压；SkyRL 的相应配置涉及准入和总容量；Miles 在设置阈值后可在消费时按最老版本过滤；verl 有 drop/wait；NeMo RL 的相关 replay 路径也有按年龄筛选。具体版本单位、默认值和 pipeline 需要分别核对。[S19][S25][S30][S36]

vLLM 的 keep 模式允许请求暂停后用新权重续写；clear_cache 控制是否丢弃旧 KV。保留旧 KV 还会带来缓存来自旧权重的问题。即使清缓存，前半段已经生成的 token 仍来自旧策略。算法数据需要保存足够来源信息。[S55]

如果按耗时淘汰长尾轨迹，困难题或复杂 agent 任务可能更容易被排除。这个偏差是调度规则与任务分布共同产生的推论，应该通过按题型、长度和年龄分层的指标验证。

5.5 低精度、MoE、packing 与数值一致性

训练与生成使用不同 kernel、精度、batch shape 或模型实现时，同一权重的 logprob 可能不同。把固定旧策略概率与真实 behavior 概率分开记录，才能区分 PPO 更新比例与训练推理偏差。[S10][S28][S54]

top-p/top-k 还改变采样支持集。若生成 logprob 和训练重算 logprob 所针对的分布不同，不能把它们当同一个分母。SkyRL 0.4.0 已有 sampler support replay，可在训练侧按同一候选集合归一化；MoE 路由 replay 又是另一项配置及数据契约。[S01]

FP8/FP4 rollout、量化 KV、MoE routing 与低精度训练应逐项加入。它们可能改善系统成本，也可能改变行为策略、数值误差或重要性比率。把所有优化同时打开，会使新算法的失败来源难以定位。

packing 要处理 sequence boundaries 和因果 attention，loss mask 要排除观察/工具返回与 padding；动态微批需要与数学目标一致的全局归一化统计量。token-mean 使用有效 token 总数，sequence-mean 则保持样本平均的定义。按样本平均、按 token 平均和固定长度归一化会改变长短回答的权重，不应当作纯系统参数。

5.6 fused/chunked loss 的三个真实边界

TRL SFT 自定义 loss。 默认 fused LM head 与自定义 full-logits 路径不同，可能额外前向计算指标。自由度增加时要确认实际执行路径。[S09]

Liger 的特定 fused PPO/GRPO 路径。 该实现提前用 autograd.grad 计算并保存梯度，反向传播主要返回 hidden/LM-head 参数梯度，advantages、old/reference logprob 作为固定量处理。这符合标准策略梯度的常见约定，但如果研究要求对 advantage 或目标生成过程继续求导，就不再是同一个计算图。不能把这一结论推广到全部 Liger kernel。[S57]

TorchTitan 的特定 ChunkedLossWrapper。 当前梯度桥接忽略外层 grad_output，因此返回 loss 后再乘常数、可学习系数或做复合张量操作都要核对。可正确分块的目标可放入 inner loss；跨块耦合或高阶求导则优先采用朴素路径，或重写该 wrapper。[S45]

保持一个小规模纯 PyTorch reference loss/step，可以在切换 kernel 后比较 loss、梯度和参数更新。对新算法来说，这往往比一开始追求最高吞吐更节省总研究时间。

6. 周边工具应该怎样使用

6.1 模型、参数化与加速

工具	有价值的职责	适合放入你的组合的位置
Transformers	模型、tokenizer、chat template 与 Trainer 生态	第一版模型基线，或 FSDP 路径的模型实现
PEFT	LoRA 等 adapter 注入与状态管理	确定要限制可更新参数子空间时使用
bitsandbytes / torchao	量化、低精度与部分内存优化	核查硬件与训练/推理一致性后按需使用
Liger / FlashAttention	融合损失与 attention 加速	新目标数值验证后逐个启用
FlashInfer	推理 kernel 与相关生成后端能力	通常由 vLLM/SGLang 集成路径使用

工具	有价值的职责	适合放入你的组合的位置
Transformer Engine / DeepEP	低精度 transformer 与 MoE 通信/执行	大模型和 MoE 的系统扩展阶段
VeOmni	模型中心、多模态、FSDP2/EP 等训练部件	多模态或自定义模型分布式训练候选
NeMo AutoModel	PyTorch DTensor 与 HF 模型训练/微调	希望保留 HF 路线并扩展模型规模时评估

这些工具减少特定执行成本，无法替你决定 reward、replay 或学习更新规则。[S57][S58][S77][S78][S79]

6.2 易用平台可以借鉴，但不必包进算法核心

LLaMA-Factory、Axolotl、ms-swift 提供大量模型、数据、模板和训练流程适配，跑标准基线很方便。它们也有真实扩展接口：例如 Axolotl 的 trainer/optimizer/collator plugin，ms-swift 的 rollout、评分与 advantage 阶段。因此“配置平台不能改算法”是不准确的。[S60][S61][S62]

对你而言，关键是修改后需要理解多少额外生命周期。一个独特 loss 若要同时经过模板、collator、Trainer mixin、缓存与运行时 patch，定位问题可能比直接在 learner 内实现更费时。

Unsloth 的优化可以缓解显存与吞吐压力，但其当前 RL 实现包含读取、改写与替换 TRL Trainer 代码的机制。研究时需要检查运行时实际类与方法，而不只是你编辑的原始文件。它更适合数值基线明确之后的优化候选。[S59]

6.3 数据、奖励、环境与评估

DataTrove 适合大规模处理、过滤和去重；distilabel 面向合成数据与 AI feedback pipeline。它们位于数据生产层，不能替代在线训练的轨迹和版本管理。[S72][S73]

Math-Verify 可作为数学答案验证工具；lm-evaluation-harness 提供统一评估入口与多种模型 backend。训练 reward、验证 reward 和报告的成功标准仍需显式定义。格式解析失败、超时、截断和执行失败应分别统计，避免隐藏在一个零分里。[S74][S75]

verifiers、SkyRL-Gym、RLinf 等在环境、agent 或具身任务上承担不同角色。环境接口的丰富度不是任意学习算法可修改性的替代指标。[S02][S38][S39]

profile 工具用于定位实际瓶颈，日志工具用于重建训练行为。优先记录可以解释算法的问题：token 数、长度分布、group reward 方差、有效梯度比例、IS ratio/clip rate、KL、entropy、critic error、样本年龄、过滤与失败原因，以及各阶段耗时。

7. 其他重要路线与维护状态

项目	定位	本报告给出的使用建议
Tunix + MaxText	JAX/Flax/Optax 的后训练与模型栈，TPU 路线重要	使用 TPU/JAX 时提高优先级；CUDA/PyTorch 经验不自动迁移
RLinf	强调具身、环境与多种模型/资源组织的 RL 基础设施	真正训练 VLA/机器人策略时重点评估
ROLL	多角色 pipeline 与多训练/推理 Strategy	任务/资源/规模要求明确时比较
Open Instruct	AllenAI 的后训练配方与实验工程	学标准配方、数据和评估如何组成完整研究
PipelineRL	持续异步、in-flight 权重更新等路线	研究异步语义与调度时值得阅读
Tinker	托管训练 API，自己写数据、loss 和 loop	适合省 GPU 运维；公开能力仍有 LoRA/模型/执行接口边界
TorchRL	PyTorch/TensorDict 的 collector、buffer、loss 等可组合部件	借鉴传统 RL 与 LLM/具身的共同数据结构
EasyR1	从 verl 分出的较聚焦多模态 RL 路线	目标模型与标准配方吻合时可快速验证；别假定等同当前 verl V1
ChatLearn	多模型执行流、资源调度与多后端平台	与 ROLL/verl 一起评估平台需求；本次仅核查官方定位
trlX	与 TRL 不同的早期 PPO/ILQL 项目	Q-head 与语言 Q-learning 的实现参考；当前依赖兼容性另行核验
ReaLHF	参数重分配与 RLHF 系统设计	官方于 2025-04-24 归档，开发迁至 AReaL；作为历史设计参考
RLHFuse / FlexFusion	论文的 intra-stage fusion 调度 artifact	适合研究 actor/critic 融合流水调度；公开内容不是完整端到端训练系统
RLHFlow	奖励建模、数据、Online-RLHF 与新算法等项目集合	按具体仓库借鉴 recipe，不能当作单一统一分布式框架
nanoRL 等极简实现	阅读与 fork 的教学/研究代码	用于理解最小闭环；大规模、MoE、恢复能力另行核查
torchtune / TinyZero	很有参考价值的旧 recipe/简化实现	官方已停止积极维护，不作为当前长期默认依赖
Cosmos-RL	早期 Physical AI RL 框架	当前官方说明有限维护、重点转向 Cosmos 3；新项目核查新体系

此表的外围项目以官方文档与代表实现为主，不代表每个后端都经过同等深度的代码审查。维护状态仅在官方明确声明时标记；新闻日期较旧本身不足以证明项目停维。[S39][S46][S63][S64][S65][S66][S67][S68][S69][S70][S71][S76][S80][S81][S82][S83][S84][S85]

Tinker 值得单独理解：它让研究者调用 forward_backward、optim_step 和 sample，把分布式训练交给服务；custom loss 可作用于返回的 logprob。这种自由度与直接持有 nn.Module、hidden state、任意 optimizer 或完整后端不同。若方法只需要受支持的概率目标，它很省事；若要改模型结构、任意优化器或全参数训练，就需要本地可控后端。[S68]

8. 建议你保留的最小算法架构

8.1 六个模块足够表达第一版

模块	自己定义的稳定职责	复用的部分
trajectory	原始 token、mask、group、版本与终止语义	框架 generator 输出，经显式 adapter 转换
algorithm	reward processing、advantage、目标与诊断	纯 PyTorch，尽量独立于分布式框架
learner	模型角色、optimizer、UTD、target 与发布规则	可使用现成 trainer；必要时由自己持有循环
rollout	单轮、多轮、工具/环境与采样	vLLM/SGLang 与框架环境/generator
runtime	分片、通信、device placement、同步	FSDP2/DeepSpeed/Megatron 与已有运行时
evaluation	固定任务、预算、原始轨迹与学习曲线	现成评估器和任务验证器

这是一份设计建议，不是已经实现并测试过的库。各模块可以先放在很少的文件里；边界稳定比目录多更重要。

8.2 概念伪代码

下面展示谁控制学习过程。接口名由你定义，不是某个框架可以直接运行的 API。

```
while not budget.exhausted():
    traces = rollout.collect(prompts, policy_version)
    records = trajectory.encode(traces)
    replay.add(records)

    for update_spec in algorithm.plan_updates(replay):
        batch = replay.sample(update_spec)
        outputs = learner.forward(batch, update_spec)
        loss, metrics = algorithm.objective(outputs, batch)
        learner.backward_and_step(loss, update_spec)
        algorithm.update_targets_if_needed(learner)

    if algorithm.should_publish():
        policy_version = runtime.publish_weights(learner)
```

如果实际方法只有单 actor 的策略梯度，把上面若干步骤交回 SkyRL/slime/TRL 等 trainer，可以减少工作。若算法具有多网络交替更新，则让 learner 暴露普通模型与 optimizer，避免把更新规则塞进本来只负责 loss 的 callback。

发布策略版本与写 checkpoint 是不同事件。一次 actor optimizer step、一次可供 rollout 使用的权重版本和一次可恢复 checkpoint，不一定是一一对应。显式命名这些时钟，可以避免异步实验里混淆 staleness 与更新次数。

9. 从实现到选择：一套具体推进顺序

9.1 第一步：把数学需求写成控制需求

用一页文档说明：需要哪些模型；哪些量可微、哪些量 detach；reward/advantage 的形状；按 token 还是按 sequence 平均；样本可重用几次；actor、critic、target 和发布各按什么频率更新；是否需要全 logits 或 hidden state；是否允许 rollout 跨版本。

这一页直接决定选框架。如果它完全落在 generator、advantage 和 policy loss 三个接口中，采用完整运行时最省事。如果涉及额外网络与不同 step lifecycle，就优先检查 worker/learner 的可替换程度。

9.2 第二步：建立一个可核对的参考实现

先选能低成本运行的模型和短上下文，以同步采样、固定精度和简单 batching 建立算法基线。保留固定的一批轨迹，比较实现与数学预期的 loss、梯度方向、关键参数更新和 mask。

这一步的目标不是冲榜，而是使每项优化之后仍有可对照的算法语义。具体模型大小由可用显存和目标模型结构决定，不依据日常桌面显卡推断研究集群。

9.3 第三步：用一个完整框架打通小规模训练

默认优先试 SkyRL FSDP2+vLLM；已有 DeepSpeed 项目则优先评估 OpenRLHF；已确定 Megatron/SGLang 则直接试 slime。离线 SFT/DPO 用 TRL 更简单。

从框架自己的、与你模型与硬件相近的完整示例开始。固定 framework commit、PyTorch/CUDA、Transformers、rollout engine 和 kernel 依赖组合。不要把所有项目的最新 main 随意混装。

先只改算法需要的一个扩展层，再验证学习曲线。若不得不多处绕过 trainer 生命周期，说明自己持有 learner 可能更清楚；这比继续堆 callback 更容易长期维护。

9.4 第四步：一次提高一种效率

按 profile 的最大开销依次处理：rollout batching/packing、reference/critic 前向、训练显存/attention、权重传输、异步重叠、低精度与 MoE 优化。每次更改重新核对与之相关的数值或数据语义。

例如同步占比高时先检查权重导出/传输/merge；环境耗时高时先增加环境并行；训练 logits 显存高时研究 chunking/fused head。单纯换一个名字更“高性能”的框架，未必解决真正瓶颈。

9.5 选择框架的小型验收实验

实验	固定什么	看什么结果
固定轨迹数值对齐	token/mask、权重、精度、目标	loss、梯度、一次参数更新与 reference 一致
单卡到多卡	有效 batch/token、目标分母、更新次数	归一化与学习行为是否保留
同步到异步	总 GPU 时间、生成/消费/更新预算	分数、年龄分布、过滤偏差、有效样本率
开启 fused/量化	同一轨迹与采样定义	梯度接口、IS ratio、KL、吞吐/显存变化
恢复与断点	数据位置、optimizer、模型/策略版本	能否按声明恢复，而不是仅载入模型权重

这些是建议的后续实验，本报告没有声称已运行。最终选择以你自己的模型、算法与 GPU 环境上的结果为准。

10. 源码阅读路线

10.1 先读算法经过的路径

建议先用一次同步迭代画出数据经过哪些函数。每到一个边界，记录张量 shape、dtype、device、mask、是否 detach、全局或局部分母、哪个 rank 执行、谁持有版本。再读异步、offload 和更高维并行。

项目	首批文件	阅读目的
SkyRL	skyrl/train/generators/base.py；skyrl/train/trainer.py；skyrl/backends/skyrl_train/utils/ppo_utils.py	数据契约、主循环、优势/损失注册
TRL	trl/trainer/grpo_trainer.py、sft_trainer.py、dpo_trainer.py；trl/generation/vllm_generation.py	生命周期、目标与权重同步
slime	train.py；slime/ray/rollout.py；slime/backends/megatron_utils/actor.py、loss.py；slime/utils/types.py	最薄 driver、可微 logits 接口、Sample
Miles	miles/backends/training_utils/loss/objective.py、hub/；miles/rollout/fully_async_data_buffer.py；miles/backends/training_utils/weight_update/updater.py	公共目标、消费策略、版本与传输
OpenRLHF	openrlhf/trainer/ppo_trainer.py、ppo_utils/experience_maker.py、ray/ppo_actor.py；openrlhf/models/loss.py	Experience、actor/critic、DeepSpeed 接线
verl V1	verl/trainer/main_ppo.py；verl/trainer/ppo/v1/trainer_base.py、replay_buffer.py；verl/trainer/ppo/core_algos.py；verl/workers/utils/losses.py	TransferQueue 数据流、注册表与 worker 边界
AReaL	areal/api/workflow_api.py；areal/trainer/rl_trainer.py；areal/trainer/ppo/actor.py、gae.py	环境、token/turn 信号与 TrainEngine 边界
NeMo RL	nemo_rl/algorithms/loss/interfaces.py；nemo_rl/models/policy/interfaces.py；nemo_rl/algorithms/grpo.py、opd.py	输入/分母契约、不同训练运行路径

项目	首批文件	阅读目的
ROLL	roll/pipeline/rlvr/rlvr_pipeline.py、actor_worker.py；roll/distributed/strategy/fsdp2_strategy.py；roll/distributed/scheduler/generate_scheduler.py	Pipeline、loss、step 与异步消费
prime-rl	src/prime_rl/orchestrator/algo/base.py；src/prime_rl/trainer/rl/loss.py、train.py；src/prime_rl/trainer/model.py	组信号、优化目标与模型范围
DeepSpeed	deepspeed/__init__.py；deepspeed/runtime/engine.py；deepspeed/runtime/zero/stage3.py	累积、分片梯度、step 和 gather
TitanRL	torchtitan/rl/types.py、trainer.py、controller.py；torchtitan/training_engine.py；torchtitan/components/loss.py	service、版本、前后向与梯度桥接

路径相对于各自仓库；slime/Miles 与 verl 的公共目录组织近期变化较大，使用后面的固定快照或 tag 查看。
[S02][S08][S09][S10][S14][S15][S18][S19][S22][S23][S24][S27][S29][S44][S45][S48]

10.2 特别值得亲自检查的代码问题

损失最后的 mean/sum 到底是对 token、sequence、rank 还是 microbatch；框架是否再做一次全局缩放。奖励标准化是否先跨 rank 聚合同一 prompt 组。top-p logprob 是否按生成支持集归一化。哪个分支偷偷把 tensor detach 或变为 numpy。

权重同步是否只发送一部分模型；量化 load 后处理是否执行；更新后 prefix/KV 是否失效。训练发布的 version 对应 optimizer step 还是发布次数。某个 ReplayBuffer 是否在 sample 后移除数据。恢复 checkpoint 是否同时恢复 optimizer、sampler、未消费数据和版本。

这些问题通常比“内置 PPO/GRPO/DPO 的数量”更直接决定你的新算法是否被忠实实现。

11. 本次审查的版本边界

项目	本次主要快照	说明
SkyRL	skyrl-v0.4.0；3d0ae22c1ba3	release 2026-10-01；固定 main 2026-10-10 01:19 UTC
TRL	72cf37df714d	2026-10-09；包含实验 AsyncGRPO
slime	v0.4.0 / 0b0c277	2026-10-09
Miles	v0.1.2 / a22fd82	2026-10-09
verl	v0.9.1 / 1876b06	2026-09-20；默认 V1
OpenRLHF	v0.11.2 / 17fd4a7	2026-09-14
ROLL	v0.4.0 / 581046abce14	2026-09-29；RLVR async 路径已实读
AReaL	298412a40f5b	2026-10-09；2.0 服务架构

项目	本次主要快照	说明
NeMo RL	d8b376aa9f48	2026-10-09；区分具体 runtime 路径
prime-rl / verifiers	f7e8ab64562e / 2cb0a6d33a49	2026-10-09
Agent Lightning	69f5a40b36d9	2026-10-09
rLLM	4e828c81680e	2026-10-09；配合能力矩阵
RLinf	0067f7d5ce52	2026-10-08
DeepSpeed	b7f51df97133	2026-10-09
TorchTitan	7fe5f405f6e0	2026-10-10 01:46 UTC；区分 core 与实验 RL
torchtune	bd2a0fc7c314	2026-04-23；停维说明
Accelerate	4ebc726849a2	2026-10-09
Liger	d5f281789aeb	2026-10-07
Megatron-Bridge	9166429859ed	2026-10-10 00:13 UTC
vLLM / SGLang	当日官方文档与 main 的同步/运行代码	滚动资料；不据此承诺任意框架依赖组合可用

核心项目审查包含多个关键实现文件，外围项目仅有部分代表文件或官方资料。支持矩阵应结合具体 model、backend、precision、LoRA、packing、parallelism 与 async 模式解释。报告的推荐也随这些约束变化。

12. 一手来源与可复核链接

以下来源对应正文的 S 编号。固定 SHA/tag 的源码是实现结论的主要依据，滚动文档用于接口解释与生态定位。各项目的性能声明只有在原有模型、硬件和配置下才有意义，本报告未将它们合并为跨框架排名。

S01 • SkyRL 统一 release 与安装依赖

github.com/NovaSky-AI/SkyRL/releases/tag/skyrl-v0.4.0
NovaSky-AI/SkyRL @skyrl-v0.4.0 • pyproject.toml

S02 • SkyRL 生成与轨迹契约

NovaSky-AI/SkyRL @3d0ae22c1ba3 • skyrl/train/generators/base.py
docs.skyrl.ai/docs/api-ref/skyrl/skyrl-train/data
docs.skyrl.ai/docs/tutorials/agent-integration

S03 • SkyRL 主循环与 fully-async

NovaSky-AI/SkyRL @3d0ae22c1ba3 • skyrl/train/trainer.py
NovaSky-AI/SkyRL @3d0ae22c1ba3 • skyrl/train/fully_async_trainer.py

S04 • SkyRL 优势与损失注册表

NovaSky-AI/SkyRL @3d0ae22c1ba3 • skyrl/backends/skyrl_train/utils/ppo_utils.py
NovaSky-AI/SkyRL @3d0ae22c1ba3 • examples/train/algorithms/custom_policy_loss/main_custom_policy_loss.py

S05 • SkyRL on-policy distillation 示例

NovaSky-AI/SkyRL @3d0ae22c1ba3 • examples/train/on_policy_distillation/main_on_policy_distill.py

S06 • SkyRL 后端与 Megatron critic 边界

docs.skyrl.ai/docs/examples/training_backends
NovaSky-AI/SkyRL @3d0ae22c1ba3 • skyrl/backends/skyrl_train/workers/megatron/megatron_worker.py

S07 • SkyRL 模型与 VLM 限制

docs.skyrl.ai/docs/getting-started/supported_models
NovaSky-AI/SkyRL @3d0ae22c1ba3 • skyrl/backends/skyrl_train/workers/model_wrapper.py
NovaSky-AI/SkyRL @3d0ae22c1ba3 • skyrl/backends/skyrl_train/workers/fsdp/fsdp_worker.py

S08 • TRL GRPOTrainer 与配置

huggingface/trl @72cf37df714d • trl/trainer/grpo_trainer.py
huggingface/trl @72cf37df714d • trl/trainer/grpo_config.py
huggingface.co/docs/trl/main/en/grpo_trainer

S09 • TRL SFTTrainer 与自定义损失

huggingface/trl @72cf37df714d • trl/trainer/sft_trainer.py
huggingface.co/docs/trl/main/en/sft_trainer

S10 • TRL 与 vLLM 的参数同步

huggingface/trl @72cf37df714d • trl/generation/vllm_generation.py

S11 • TRL 实验 AsyncGRPO

huggingface.co/docs/trl/main/en/async_grpo_trainer
huggingface/trl @72cf37df714d • docs/source/async_grpo_trainer.md

S12 • TRL DPOTrainer

huggingface/trl @72cf37df714d • trl/trainer/dpo_trainer.py

S13 • slime 当前 release 与 FSDP 移除

github.com/THUDM/slime/releases/tag/v0.4.0
github.com/THUDM/slime/releases/tag/v0.2.3

S14 · slime driver 与 actor

THUDM/slime @v0.4.0 · train.py
THUDM/slime @v0.4.0 · slime/ray/rollout.py
THUDM/slime @v0.4.0 · slime/backends/megatron_utils/actor.py

S15 · slime 目标、优势与 reducer

THUDM/slime @v0.4.0 · slime/backends/megatron_utils/loss.py
THUDM/slime @v0.4.0 · slime/utils/ppo_utils.py

S16 · slime Sample 与异步生成

THUDM/slime @v0.4.0 · slime/utils/types.py
THUDM/slime @v0.4.0 · slime/rollout/fully_async_rollout.py

S17 · Miles v0.1.2

github.com/radixark/miles/releases/tag/v0.1.2

S18 · Miles 公共 loss 与 advantage

radixark/miles @v0.1.2 · miles/backends/training_utils/loss/objective.py
radixark/miles @v0.1.2 · miles/backends/training_utils/loss/hub/losses.py
radixark/miles @v0.1.2 · miles/backends/training_utils/loss/hub/advantages.py

S19 · Miles 异步 buffer 与版本消费

radixark/miles @v0.1.2 · miles/rollout/fully_async_data_buffer.py
radixark/miles @v0.1.2 · miles/rollout/fully_async_rollout.py
radixark/miles @v0.1.2 · docs/user-guide/fully-async.md

S20 · Miles WeightUpdater 与传输协议

radixark/miles @v0.1.2 · miles/backends/training_utils/weight_update/updater.py
radixark/miles @v0.1.2 · miles/backends/training_utils/weight_update/protocol.py

S21 · verl v0.9.1 与 V1 切换

github.com/verl-project/verl/releases/tag/v0.9.1
github.com/verl-project/verl/releases/tag/v0.9.0

S22 · verl V1 主循环与数据流

verl-project/verl @v0.9.1 · verl/trainer/main_ppo.py
verl-project/verl @v0.9.1 · verl/trainer/ppo/v1/trainer_base.py

S23 · verl 算法注册表

verl-project/verl @v0.9.1 · verl/trainer/ppo/core_algos.py

S24 · verl worker 的 policy-loss 契约

verl-project/verl @v0.9.1 · verl/workers/utils/losses.py

S25 · verl V1 buffer、drop 与 wait

verl-project/verl @v0.9.1 · verl/trainer/ppo/v1/replay_buffer.py
verl-project/verl @v0.9.1 · verl/trainer/ppo/v1/trainer_separate_async.py

S26 · OpenRLHF v0.11.2

github.com/OpenRLHF/OpenRLHF/releases/tag/v0.11.2

S27 · OpenRLHF Experience 与估计器

OpenRLHF/OpenRLHF @v0.11.2 · openrlhf/trainer/ppo_utils/experience.py
OpenRLHF/OpenRLHF @v0.11.2 · openrlhf/trainer/ppo_utils/experience_maker.py

S28 · OpenRLHF PolicyLoss

OpenRLHF/OpenRLHF @v0.11.2 · openrlhf/models/loss.py

S29 · OpenRLHF actor update 与权重广播

OpenRLHF/OpenRLHF @v0.11.2 · openrlhf/trainer/ray/ppo_actor.py

S30 · OpenRLHF 同步与异步主循环

OpenRLHF/OpenRLHF @v0.11.2 · openrlhf/trainer/ppo_trainer.py
OpenRLHF/OpenRLHF @v0.11.2 · openrlhf/trainer/ppo_trainer_async.py

S31 · AReaL 2.0 架构与生态

areal-project/AReaL @298412a40f5b · README.md

S32 · AReaL learner、优势与多教师损失

areal-project/AReaL @298412a40f5b · areal/trainer/rl_trainer.py
areal-project/AReaL @298412a40f5b · areal/trainer/ppo/actor.py
areal-project/AReaL @298412a40f5b · areal/trainer/ppo/gae.py
areal-project/AReaL @298412a40f5b · areal/trainer/mopd/loss.py

S33 · AReaL workflow 与权重服务

areal-project/AReaL @298412a40f5b · areal/api/workflow_api.py
areal-project/AReaL @298412a40f5b · areal/v2/weight_update/controller/controller.py

S34 · NeMo RL 与蒸馏运行路径

NVIDIA-NeMo/RL @d8b376aa9f48 · README.md
NVIDIA-NeMo/RL @d8b376aa9f48 · nemo_rl/algorithms/distillation.py
NVIDIA-NeMo/RL @d8b376aa9f48 · nemo_rl/algorithms/opd.py

S35 · NeMo RL PolicyInterface 与 LossFunction

NVIDIA-NeMo/RL @d8b376aa9f48 · nemo_rl/models/policy/interfaces.py
NVIDIA-NeMo/RL @d8b376aa9f48 · nemo_rl/algorithms/loss/interfaces.py

S36 · NeMo RL 异步 GRPO、buffer 与 refit

NVIDIA-NeMo/RL @d8b376aa9f48 · nemo_rl/algorithms/grpo.py
NVIDIA-NeMo/RL @d8b376aa9f48 · nemo_rl/algorithms/async_utils/replay_buffer.py
NVIDIA-NeMo/RL @d8b376aa9f48 · nemo_rl/algorithms/async_utils/trajectory_collector.py

S37 · prime-rl：信号、目标、模型与训练队列

PrimeIntellect-ai/prime-rl @f7e8ab64562e · src/prime_rl/orchestrator/algo/base.py
PrimeIntellect-ai/prime-rl @f7e8ab64562e · src/prime_rl/trainer/rl/loss.py
PrimeIntellect-ai/prime-rl @f7e8ab64562e · src/prime_rl/trainer/model.py
PrimeIntellect-ai/prime-rl @f7e8ab64562e · src/prime_rl/trainer/rl/train.py
PrimeIntellect-ai/prime-rl @f7e8ab64562e · README.md

S38 · verifiers 环境与训练轨迹

PrimeIntellect-ai/verifiers @2cb0a6d33a49 · README.md

S39 · RLinf Worker/Channel、registry 与 runner

RLinf/RLinf @0067f7d5ce52 · rlinf/algorithms/registry.py
RLinf/RLinf @0067f7d5ce52 · rlinf/runners/async_embodied_runner.py
rlinf.readthedocs.io/en/latest/rst_source/concepts/worker.html

S40 · Agent Lightning 轨迹与共享 GPU 异步

microsoft/agent-lightning @69f5a40b36d9 · agentlightning/verl/rollout_adapter.py
microsoft/agent-lightning @69f5a40b36d9 · agentlightning/verl/trainer.py
microsoft/agent-lightning @69f5a40b36d9 · docs/35-asynchronous-training.md
microsoft.github.io/agent-lightning/stable/35-asynchronous-training/

S41 · rLLM 后端能力矩阵

docs.rllm-project.com/training/capability-matrix
rllm-org/rllm @4e828c81680e · rllm/trainer/unified_trainer.py

S42 • TorchTitan core 与发布范围

pytorch/torchtitan @7fe5f405f6e0 • README.md
github.com/pytorch/torchtitan/releases/tag/v0.3.0

S43 • TitanRL 范围、模型与数值一致性

pytorch/torchtitan @7fe5f405f6e0 • torchtitan/rl/README.md
pytorch/torchtitan @7fe5f405f6e0 • torchtitan/rl/docs/bitwise_parity.md

S44 • TitanRL 训练服务与数据协议

pytorch/torchtitan @7fe5f405f6e0 • torchtitan/rl/trainer.py
pytorch/torchtitan @7fe5f405f6e0 • torchtitan/rl/types.py
pytorch/torchtitan @7fe5f405f6e0 • torchtitan/rl/controller.py
pytorch/torchtitan @7fe5f405f6e0 • torchtitan/rl/generator.py

S45 • TorchTitan ChunkedLossWrapper 梯度边界

pytorch/torchtitan @7fe5f405f6e0 • torchtitan/components/loss.py

S46 • torchtune 停止积极维护说明

meta-pytorch/torchtune @bd2a0fc7c314 • README.md

S47 • DeepSpeed 初始化与训练 API

deepspeed.readthedocs.io/en/latest/initialize.html
deepspeed.readthedocs.io/en/latest/training.html
deepspeedai/DeepSpeed @b7f51df97133 • deepspeed/__init__.py

S48 • DeepSpeedEngine 与 ZeRO3

deepspeedai/DeepSpeed @b7f51df97133 • deepspeed/runtime/engine.py
deepspeedai/DeepSpeed @b7f51df97133 • deepspeed/runtime/zero/stage3.py

S49 • DeepSpeed-Chat PPO 示例

deepspeedai/DeepSpeedExamples @0c6dd5ed1ec0 • applications/DeepSpeed-Chat/dschat/rlhf/ppo_trainer.py

S50 • PyTorch FSDP2 官方接口与教程

docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html
docs.pytorch.org/docs/stable/distributed.fsdp.fully_shard.html

S51 • Megatron-Core 与 pipeline schedule

github.com/NVIDIA/Megatron-LM
NVIDIA/Megatron-LM @main • megatron/core/pipeline_parallel/schedules.py

S52 • Megatron-Bridge 训练入口与 forward step

NVIDIA-NeMo/Megatron-Bridge @9166429859ed • src/megatron/bridge/training/finetune.py
NVIDIA-NeMo/Megatron-Bridge @9166429859ed • src/megatron/bridge/training/train.py
NVIDIA-NeMo/Megatron-Bridge @9166429859ed • src/megatron/bridge/training/gpt_step.py

S53 • Accelerate prepare 与 FSDP2

huggingface/accelerate @4ebc726849a2 • src/accelerate/accelerator.py
huggingface/accelerate @4ebc726849a2 • src/accelerate/utils/fsdp_utils.py

S54 • vLLM 原生 RL 权重传输

docs.vllm.ai/en/latest/training/weight_transfer/
vllm-project/vllm @main • vllm/distributed/weight_transfer/base.py
vllm-project/vllm @main • vllm/distributed/weight_transfer/factory.py

S55 • vLLM in-flight generation 与缓存语义

vllm-project/vllm @main • vllm/v1/engine/async_llm.py

S56 • SGLang RL 与权重更新 session

sgl-project/sglang @main • docs/docs/advanced_features/sglang_for_rl.mdx

sgl-project/sglang @main · python/sglang/srt/model_executor/model_runner_components/weight_updater.py

S57 · Liger 特定 fused PPO 的梯度实现

linkedin/Liger-Kernel @d5f281789aeb · src/liger_kernel/chunked_loss/fused_linear_ppo.py

linkedin/Liger-Kernel @d5f281789aeb · src/liger_kernel/chunked_loss/grpo_loss.py

S58 · PEFT 参数高效微调

github.com/huggingface/peft

S59 · Unsloth 对 RL Trainer 的运行时修改

unslothai/unsloth @main · unsloth/models/rl.py

S60 · Axolotl plugin 接口

docs.axolotl.ai/docs/api/integrations.base.html

axolotl-ai-cloud/axolotl @main · src/axolotl/integrations/base.py

S61 · LLaMA-Factory DPOTrainer

hiyouga/LlamaFactory @main · src/llamafactory/train/dpo/trainer.py

S62 · ms-swift GRPO 生命周期与 Liger 路径

modelscope/ms-swift @main · swift/rlhf_trainers/grpo_trainer.py

S63 · Tunix JAX 后训练

github.com/google/tunix

S64 · MaxText JAX 模型与训练栈

github.com/AI-Hypercomputer/maxtext

S65 · Cosmos-RL 维护与迁移说明

github.com/nvidia-cosmos/cosmos-rl

S66 · PipelineRL 异步训练

github.com/ServiceNow/PipelineRL

S67 · AllenAI Open Instruct

github.com/allenai/open-instruct

S68 · Tinker API 与 custom loss

tinker-docs.thinkingmachines.ai/

tinker-docs.thinkingmachines.ai/tinker/losses/custom/

S69 · TorchRL 可组合 RL 基础设施

github.com/pytorch/rl

S70 · TinyZero 维护状态

github.com/Jiayi-Pan/TinyZero

S71 · 极简 nanoRL 示例（同名项目较多）

github.com/alex000kim/nanoRL

S72 · DataTrove 数据处理

github.com/huggingface/datatrove

S73 · distilabel 合成数据与 AI feedback

github.com/argilla-io/distilabel

S74 · Math-Verify 数学答案验证

github.com/huggingface/Math-Verify

S75 · Language Model Evaluation Harness

github.com/EleutherAI/lm-evaluation-harness

S76 · ROLL v0.4.0：pipeline、loss、Strategy 与调度

github.com/alibaba/ROLL/releases/tag/v0.4.0
alibaba/ROLL @v0.4.0 · roll/pipeline/rlvr/rlvr_pipeline.py
alibaba/ROLL @v0.4.0 · roll/pipeline/rlvr/actor_worker.py
alibaba/ROLL @v0.4.0 · roll/pipeline/base_worker.py
alibaba/ROLL @v0.4.0 · roll/distributed/strategy/strategy.py
alibaba/ROLL @v0.4.0 · roll/distributed/strategy/fsdp2_strategy.py
alibaba/ROLL @v0.4.0 · roll/distributed/scheduler/generate_scheduler.py
alibaba/ROLL @v0.4.0 · roll/utils/functionals.py

S77 · VeOmni 多模态训练底座

github.com/ByteDance-Seed/VeOmni

S78 · NeMo AutoModel

github.com/NVIDIA-NeMo/Automodel

S79 · Kernel、低精度与 MoE 通信组件

github.com/Dao-AILab/flash-attention
github.com/flashinfer-ai/flashinfer
github.com/NVIDIA/TransformerEngine
github.com/deepseek-ai/DeepEP
github.com/pytorch/ao
github.com/bitsandbytes-foundation/bitsandbytes

S80 · EasyR1 聚焦多模态 RL

github.com/hiyouga/EasyR1

S81 · ReaLHF 归档与迁移说明

github.com/openpsi-project/ReaLHF

S82 · ChatLearn 多模型执行流平台

github.com/alibaba/ChatLearn

S83 · trlX PPO/ILQL 代码参考

github.com/CarperAI/trlx

S84 · RLHFuse / FlexFusion 公开 artifact 范围

github.com/FlexFusion/FlexFusion

S85 · RLHFlow 奖励建模与在线 RL recipes

github.com/RLHFlow/RLHF-Reward-Modeling
github.com/RLHFlow/Online-RLHF